

Cinematica per Robot ed Agenti Autonomi

NECKER

2 Giugno 2026

Indice

1	Cosa è la Cinematica	2
2	Vettori, Matrici e Sistemi di Riferimento	2
2.1	Sistemi di Riferimento (Frames)	2
2.2	Vettori e Matrici	2
2.3	Dot Product	3
2.4	Cross Product	3
3	Definizione Formale di un Frame e Matrici di Rotazione	4
3.1	Un frame rispetto ad un altro: La Matrice di Rotazione	5
4	Cambio di Coordinate (Pose)	6
5	Rotazioni Composte	6
5.1	Rotazioni Elementari	6
5.2	Regola di Composizione	6
6	Parametrizzazioni dell'Orientamento	6
6.1	Angoli di Eulero (Z-Y-Z) e Roll-Pitch-Yaw	6
6.2	Quaternioni	6
7	Matrici di Trasformazione Omogenee	8
8	Alberi di Trasformazione (TF Trees)	8
9	Tabella Riassuntiva delle Rappresentazioni	9

1 Cosa è la Cinematica

I robot sono corpi rigidi che si muovono in spazi tridimensionali reali. Affinché un algoritmo possa prendere decisioni sull'attuazione e deliberare sul movimento, necessita di una rappresentazione matematica rigorosa.

La **cinematica** è il ramo della meccanica che studia il moto dei corpi ignorando le cause (forze e coppie) che lo generano. Si concentra esclusivamente sulla geometria del movimento, descrivendo grandezze come posizione, velocità e accelerazione.

2 Vettori, Matrici e Sistemi di Riferimento

2.1 Sistemi di Riferimento (Frames)

Il concetto centrale della cinematica è il sistema di riferimento (o *frame*). Si assume l'esistenza di almeno due frame fondamentali:

- **World frame (Sistema Globale):** Un sistema di riferimento assoluto in cui vengono specificate tutte le coordinate dell'ambiente.
- **Robot frame (Sistema Locale):** Un frame rigidamente attaccato al robot. Essendo il robot considerato un *corpo rigido* (un oggetto in cui la distanza euclidea tra due punti qualsiasi non cambia mai), il frame si muove solidalmente con esso.

2.2 Vettori e Matrici

Un **vettore geometrico** v è un'entità definita da magnitudo (lunghezza), direzione e verso, indipendente dal sistema di riferimento. Quando associato a un frame, diventa un vettore di coordinate $\mathbf{p} \in \mathbb{R}^n$, ovvero una lista ordinata di numeri.

Una **matrice** è un array rettangolare di scalari di dimensione $m \times n$. Il **prodotto tra matrici** (non commutativo) tra A ($m \times k$) e B ($k \times n$) restituisce C ($m \times n$) tale che:

$$c_{i,j} = \sum_k a_{i,k} b_{k,j}$$

Altre operazioni fondamentali:

- **Trasposta** (A^T): Ottenuta scambiando righe con colonne.
- **Inversa** (A^{-1}): Matrice tale che $AA^{-1} = I$, esistente solo per matrici quadrate con determinante non nullo.

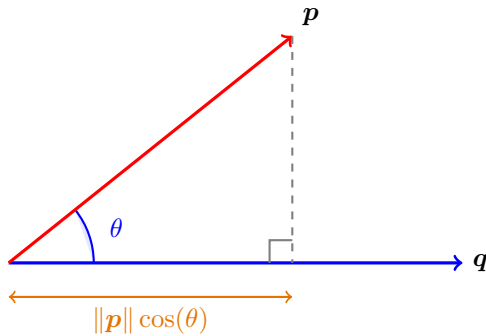
2.3 Dot Product

Il prodotto scalare tra due vettori $\mathbf{p}$ e $\mathbf{q}$ restituisce uno scalare che rappresenta la lunghezza della proiezione di $\mathbf{p}$ su $\mathbf{q}$, moltiplicata per la lunghezza di $\mathbf{q}$:

$$\mathbf{p} \cdot \mathbf{q} = \|\mathbf{p}\| \|\mathbf{q}\| \cos(\theta)$$

- $\mathbf{p} \cdot \mathbf{q} > 0$: direzioni generalmente concordi ($\theta < 90^\circ$).
- $\mathbf{p} \cdot \mathbf{q} = 0$: vettori ortogonali ($\theta = 90^\circ$).
- $\mathbf{p} \cdot \mathbf{q} < 0$: direzioni opposte ($\theta > 90^\circ$).

Sotto forma matriciale: $\mathbf{p} \cdot \mathbf{q} = \mathbf{p}^T \mathbf{q} = p_x q_x + p_y q_y + p_z q_z$.



$$\mathbf{p} \cdot \mathbf{q} = \|\mathbf{p}\| \|\mathbf{q}\| \cos(\theta)$$

2.4 Cross Product

Il prodotto vettoriale tra due vettori $\mathbf{p}, \mathbf{q} \in \mathbb{R}^3$ è un'operazione che produce un nuovo vettore, indicato con $\mathbf{p} \times \mathbf{q}$, avente le seguenti proprietà fondamentali:

- è ortogonale sia a $\mathbf{p}$ sia a $\mathbf{q}$;
- il suo verso è determinato dalla regola della mano destra;
- il suo modulo è pari all'area del parallelogramma generato dai due vettori.

In particolare,

$$\|\mathbf{p} \times \mathbf{q}\| = \|\mathbf{p}\| \|\mathbf{q}\| \sin \theta$$

dove θ è l'angolo compreso tra $\mathbf{p}$ e $\mathbf{q}$. (base * altezza)

Se i due vettori sono paralleli (o antiparalleli), allora $\sin \theta = 0$ e quindi

$$\mathbf{p} \times \mathbf{q} = \mathbf{0}.$$

Questo riflette il fatto che il parallelogramma degeneri in un segmento, avente area nulla.

Computazionalmente

Il prodotto vettoriale tra due vettori $\mathbf{p}$ e $\mathbf{q}$ si calcola direttamente componente per componente come:

$$\mathbf{p} \times \mathbf{q} = \begin{bmatrix} p_y q_z - p_z q_y \\ p_z q_x - p_x q_z \\ p_x q_y - p_y q_x \end{bmatrix}.$$

Questa è la forma esplicita del prodotto vettoriale e deriva direttamente dalla sua definizione geometrica: il risultato è un vettore perpendicolare a $\mathbf{p}$ e $\mathbf{q}$, con modulo pari all'area del parallelogramma da essi generato.

Lo stesso risultato può però essere ottenuto in modo più compatto introducendo una matrice associata al vettore $\mathbf{p}$, detta matrice emisimmetrica ($T = -T^T$):

$$[\mathbf{p}_\times] = \begin{bmatrix} 0 & -p_z & p_y \\ p_z & 0 & -p_x \\ -p_y & p_x & 0 \end{bmatrix}.$$

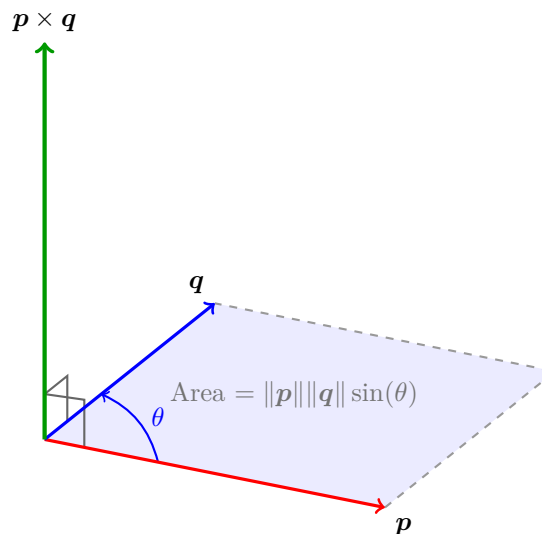
Moltiplicando questa matrice per il vettore $\mathbf{q}$ si ottiene:

$$[\mathbf{p}_\times]\mathbf{q} = \begin{bmatrix} 0 & -p_z & p_y \\ p_z & 0 & -p_x \\ -p_y & p_x & 0 \end{bmatrix} \begin{bmatrix} q_x \\ q_y \\ q_z \end{bmatrix}.$$

Svolgendo il prodotto matrice-vettore si recuperano esattamente le stesse componenti:

$$[\mathbf{p}_\times]\mathbf{q} = \begin{bmatrix} p_y q_z - p_z q_y \\ p_z q_x - p_x q_z \\ p_x q_y - p_y q_x \end{bmatrix} = \mathbf{p} \times \mathbf{q}.$$

Quindi la matrice emisimmetrica non introduce una nuova operazione, ma è semplicemente una riscrittura del prodotto vettoriale che permette di trattarlo come una moltiplicazione lineare tra matrice e vettore.



3 Definizione Formale di un Frame e Matrici di Rotazione

Prima di descrivere il movimento, dobbiamo definire rigorosamente l'ambiente in cui esso avviene. In cinematica, lo spazio è descritto tramite **Sistemi di Riferimento** (o *Frame*).

Un frame, denotato convenzionalmente come $O - \mathbf{xyz}$, non è altro che un'impalcatura matematica definita da due elementi essenziali:

1. Un punto di origine O , che fissa la posizione del sistema nello spazio (la radice da cui partono le misurazioni di distanza).
2. Una terna di **versori** (vettori di lunghezza unitaria) $\mathbf{x}, \mathbf{y}, \mathbf{z}$, che definiscono le direzioni degli assi coordinati e, conseguentemente, l'orientamento del frame.

Affinché questa terna formi uno spazio cartesiano standard e fisicamente coerente, i tre versori non possono essere scelti a caso, ma devono obbedire a due rigide leggi geometriche: l'ortonormalità e la regola della mano destra.

1. Il Vincolo di Ortonormalità

Gli assi devono essere perpendicolari tra loro (ortogonali) e la scala di misura deve essere unitaria (normali). In linguaggio matematico, utilizzando il prodotto scalare, questo si traduce nel sistema:

$$\begin{cases} \mathbf{x} \cdot \mathbf{x} = 1, & \mathbf{y} \cdot \mathbf{y} = 1, & \mathbf{z} \cdot \mathbf{z} = 1 & \text{(Normalità: i vettori hanno lunghezza 1)} \\ \mathbf{x} \cdot \mathbf{y} = 0, & \mathbf{x} \cdot \mathbf{z} = 0, & \mathbf{y} \cdot \mathbf{z} = 0 & \text{(Ortogonalità: gli assi sono a 90° tra loro)} \end{cases}$$

Questa proprietà garantisce che i tre assi rappresentino gradi di libertà spaziali completamente indipendenti: muoversi lungo x non comporta alcun movimento lungo y o z .

2. La Regola della Mano Destra (Chiralità dello Spazio)

Non basta che gli assi siano a 90° . Esistono due modi speculari per disporre tre assi ortogonali (uno destrorso e uno sinistrorso). In robotica e fisica si adotta la *regola della mano destra* per garantire che il calcolo vettoriale (come le rotazioni e i momenti torcenti) mantenga un segno coerente. Questo vincolo si impone attraverso il prodotto vettoriale:

$$\mathbf{x} \times \mathbf{y} = \mathbf{z}, \quad \mathbf{y} \times \mathbf{z} = \mathbf{x}, \quad \mathbf{z} \times \mathbf{x} = \mathbf{y}$$

Se il prodotto $\mathbf{x} \times \mathbf{y}$ restituisse $-\mathbf{z}$, avremmo costruito un sistema sinistrorso (come l'immagine riflessa in uno specchio di un sistema standard), rendendo nulle tutte le convenzioni cinematiche standard.

3.1 Un frame rispetto ad un altro: La Matrice di Rotazione

Nella realtà, un robot ha molteplici frame (uno per ogni giunto, uno per la base, uno per la telecamera). Consideriamo il problema di descrivere l'orientamento di un frame locale $B = O' - \mathbf{x}'\mathbf{y}'\mathbf{z}'$ (ad esempio, la mano del robot) visto dagli "occhi" di un frame globale $A = O - \mathbf{x}\mathbf{y}\mathbf{z}$ (la stanza).

Come facciamo a descrivere l'inclinazione di B rispetto ad A ? La strategia è semplice: **osserviamo i tre assi di B e li misuriamo usando i "metri" del frame A .**

Per farlo, dobbiamo prendere ciascun asse di B (ovvero $\mathbf{x}', \mathbf{y}', \mathbf{z}'$) e proiettarlo ortogonalmente sugli assi di A ($\mathbf{x}, \mathbf{y}, \mathbf{z}$). Lo strumento matematico perfetto per proiettare un vettore su un altro è il **prodotto scalare**.

Prendiamo ad esempio il primo asse di B , ovvero $\mathbf{x}'$. Le sue coordinate lette nel frame A saranno:

$${}^A\mathbf{x}' = \begin{bmatrix} \mathbf{x}' \cdot \mathbf{x} \\ \mathbf{x}' \cdot \mathbf{y} \\ \mathbf{x}' \cdot \mathbf{z} \end{bmatrix}$$

Se ripetiamo questo procedimento per tutti e tre gli assi di B e li affianchiamo come colonne, otteniamo un'unica struttura matematica chiamata **Matrice di Rotazione** ${}^A R_B$:

$${}^A R_B = \begin{bmatrix} \uparrow & \uparrow & \uparrow \\ {}^A\mathbf{x}' & {}^A\mathbf{y}' & {}^A\mathbf{z}' \\ \downarrow & \downarrow & \downarrow \end{bmatrix} = \begin{bmatrix} \mathbf{x}' \cdot \mathbf{x} & \mathbf{y}' \cdot \mathbf{x} & \mathbf{z}' \cdot \mathbf{x} \\ \mathbf{x}' \cdot \mathbf{y} & \mathbf{y}' \cdot \mathbf{y} & \mathbf{z}' \cdot \mathbf{y} \\ \mathbf{x}' \cdot \mathbf{z} & \mathbf{y}' \cdot \mathbf{z} & \mathbf{z}' \cdot \mathbf{z} \end{bmatrix}$$

Il Significato Profondo: I Coseni Direttori

C'è un dettaglio fondamentale da notare: poiché tutti i vettori coinvolti ($\mathbf{x}, \mathbf{y}, \mathbf{z}$ e $\mathbf{x}', \mathbf{y}', \mathbf{z}'$) sono *versori* (hanno lunghezza 1), il loro prodotto scalare si semplifica enormemente. Ricordando la definizione trigonometrica del prodotto scalare ($\mathbf{v} \cdot \mathbf{w} = \|\mathbf{v}\|\|\mathbf{w}\|\cos\theta$), otteniamo che:

$$\mathbf{x}' \cdot \mathbf{x} = (1)(1)\cos(\theta_{\mathbf{x}'\mathbf{x}}) = \cos(\theta_{\mathbf{x}'\mathbf{x}})$$

Ogni singolo elemento della matrice di rotazione non è altro che il **coseno dell'angolo** formato tra un asse del frame B e un asse del frame A .

NOTARE: alla fine una rotazione consiste nel trovare dove si trova la punta del vettore (si parte con i vettori dei tre assi che poi si spostano a causa della rotazione), quindi si usano le componenti x e y (trovate con seno e coseno) per individuare la nuova posizione

graficamente: <https://www.youtube.com/watch?v=-pQtkHEoueE>

4 Cambio di Coordinate (Pose)

La *Posa* di un robot è l'unione della sua posizione (origine ${}^A O'$) e del suo orientamento (${}^A R_B$). Dato un punto ${}^B \mathbf{p}$ nel frame B , per trovare le sue coordinate nel frame globale A , sommiamo la traslazione alla rotazione del vettore:

$${}^A \mathbf{p} = {}^A O' + {}^A R_B {}^B \mathbf{p}$$

5 Rotazioni Composte

5.1 Rotazioni Elementari

Le rotazioni attorno agli assi principali di un angolo α sono descritte da:

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix}, \quad R_y(\alpha) = \begin{bmatrix} \cos \alpha & 0 & \sin \alpha \\ 0 & 1 & 0 \\ -\sin \alpha & 0 & \cos \alpha \end{bmatrix}, \quad R_z(\alpha) = \begin{bmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

5.2 Regola di Composizione

Quando si applicano rotazioni in sequenza $R_1, R_2, \dots, R_n$:

- Se le rotazioni avvengono rispetto agli **assi fissi** (globali), si **pre-moltiplica**: ${}^A R_B = R_n \dots R_2 R_1$.
- Se le rotazioni avvengono rispetto agli **assi mobili** (locali), si **post-moltiplica**: ${}^A R_B = R_1 R_2 \dots R_n$.

Inoltre, le matrici concatenate permettono cancellazioni dei pedici/apici: ${}^A R_B {}^B R_C = {}^A R_C$.

6 Parametrizzazioni dell'Orientamento

Le matrici di rotazione usano 9 parametri per descrivere 3 gradi di libertà. Sono *ridondanti*. Esistono rappresentazioni minimali:

6.1 Angoli di Eulero (Z-Y-Z) e Roll-Pitch-Yaw

- **Roll-Pitch-Yaw**: Rotazioni rispettivamente su X , Y e Z degli **assi fissi**.
- **Angoli di Eulero Z-Y-Z**: Tre rotazioni successive attorno agli **assi mobili**.

Il problema principale delle parametrizzazioni a 3 angoli sono le **Singularità (Gimbal Lock)**. Ad esempio, in configurazione Z-Y-Z, se l'angolo intermedio $\beta = \pi$ o $\beta = 0$, il primo e l'ultimo asse di rotazione si allineano. Questo causa la perdita di un grado di libertà (il sistema "collapsa") e impedisce il calcolo del problema inverso. È analogo a trovarsi al Polo Nord terrestre: la longitudine perde significato matematico.

6.2 Quaternioni

Per superare le limitazioni delle matrici di rotazione e degli angoli di Eulero — in particolare il problema del *Gimbal Lock* — si ricorre all'uso dei **Quaternioni**. Introdotti dal matematico William Rowan Hamilton, essi rappresentano un'estensione dei numeri complessi in uno spazio a quattro dimensioni.

Un quaternion $\mathbf{q} \in \mathbb{H}$ è definito come una combinazione lineare di uno scalare reale e tre unità immaginarie i, j, k :

$$\mathbf{q} = q_0 + q_1 i + q_2 j + q_3 k$$

dove i coefficienti $q_0, q_1, q_2, q_3 \in \mathbb{R}$, e le unità immaginarie soddisfano le relazioni fondamentali:

$$i^2 = j^2 = k^2 = ijk = -1$$

Spesso, risulta utile esprimere il quaternion separando la sua parte scalare da quella vettoriale: $\mathbf{q} = (q_0, \mathbf{v})$, con $\mathbf{v} = [q_1 \quad q_2 \quad q_3]^T$.

Per descrivere le rotazioni nello spazio tridimensionale, si utilizzano esclusivamente i **Quaternioni Unitari** (ovvero con norma $\|\mathbf{q}\| = 1$). In accordo con il **Teorema di Rotazione di Eulero**, qualsiasi rotazione 3D

può essere descritta tramite un angolo θ attorno a un asse di rotazione arbitrario, individuato da un versore $\mathbf{r} = [r_x, r_y, r_z]^T$. Il quaternione unitario associato a tale rotazione è formulato come:

$$\mathbf{q} = \cos\left(\frac{\theta}{2}\right) + \sin\left(\frac{\theta}{2}\right)(r_x i + r_y j + r_z k)$$

L'utilizzo dell'angolo dimezzato $\frac{\theta}{2}$ deriva dalla natura dell'operatore di rotazione. Per applicare la rotazione a un punto o a un vettore $\mathbf{p}$ nello spazio (rappresentato come un quaternione puro $\mathbf{p} = 0 + p_x i + p_y j + p_z k$), si utilizza il prodotto quaternionico tramite il coniugato:

$$\mathbf{p}' = \mathbf{q} \otimes \mathbf{p} \otimes \mathbf{q}^*$$

dove $\mathbf{q}^*$ è il coniugato del quaternione ($\mathbf{q}^* = q_0 - \mathbf{v}$) e $\otimes$ denota il prodotto di Hamilton. La doppia moltiplicazione applica due volte la rotazione di $\frac{\theta}{2}$, risultando in una rotazione netta esattamente di θ .

Esempio Pratico di Rotazione:

Si consideri di voler ruotare un vettore $\mathbf{v}_{in} = [0 \ 1 \ 0]^T$ (situato lungo l'asse y) di un angolo $\theta = 90^\circ$ ($\frac{\pi}{2}$ radianti) attorno all'asse z , identificato dal versore $\mathbf{r} = [0 \ 0 \ 1]^T$. Geometricamente, ci si aspetta che il vettore ruotato si trovi sull'asse x negativo, ovvero $\mathbf{v}_{out} = [-1 \ 0 \ 0]^T$.

1. **Costruzione del quaternione di rotazione $\mathbf{q}$:** Essendo $\frac{\theta}{2} = 45^\circ$ ($\frac{\pi}{4}$ rad), si ha $\cos(\frac{\pi}{4}) = \frac{\sqrt{2}}{2}$ e $\sin(\frac{\pi}{4}) = \frac{\sqrt{2}}{2}$. Pertanto:

$$\mathbf{q} = \frac{\sqrt{2}}{2} + 0i + 0j + \frac{\sqrt{2}}{2}k = \frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}k$$

Il suo coniugato $\mathbf{q}^*$ è:

$$\mathbf{q}^* = \frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}k$$

2. Codifica del vettore come quaternione puro $\mathbf{p}$:

$$\mathbf{p} = 0 + 0i + 1j + 0k = j$$

3. **Calcolo del primo prodotto $\mathbf{q} \otimes \mathbf{p}$:** Sfruttando le regole distributive e sapendo che $k \otimes j = -i$, si ottiene:

$$\mathbf{q} \otimes \mathbf{p} = \left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}k\right) \otimes j = \frac{\sqrt{2}}{2}j + \frac{\sqrt{2}}{2}(k \otimes j) = -\frac{\sqrt{2}}{2}i + \frac{\sqrt{2}}{2}j$$

4. **Calcolo del prodotto finale $\mathbf{p}' = (\mathbf{q} \otimes \mathbf{p}) \otimes \mathbf{q}^*$:** Moltiplicando il risultato precedente per il coniugato $\mathbf{q}^*$ e ricordando le identità dei prodotti ($i \otimes k = -j$ e $j \otimes k = i$), si sviluppano i termini:

$$\mathbf{p}' = \left(-\frac{\sqrt{2}}{2}i + \frac{\sqrt{2}}{2}j\right) \otimes \left(\frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}k\right)$$

$$\mathbf{p}' = -\frac{2}{4}i + \frac{2}{4}(i \otimes k) + \frac{2}{4}j - \frac{2}{4}(j \otimes k)$$

$$\mathbf{p}' = -\frac{1}{2}i + \frac{1}{2}(-j) + \frac{1}{2}j - \frac{1}{2}(i)$$

$$\mathbf{p}' = -\frac{1}{2}i - \frac{1}{2}j + \frac{1}{2}j - \frac{1}{2}i = -1i$$

Il quaternione risultante è $\mathbf{p}' = 0 - 1i + 0j + 0k$. Estraeendo la parte vettoriale, si ottiene esattamente il vettore cercato: $\mathbf{v}_{out} = [-1 \ 0 \ 0]^T$.

7 Matrici di Trasformazione Omogenee

Nello spazio tridimensionale euclideo, una trasformazione lineare (come una rotazione) può essere espressa tramite una matrice 3×3 . Tuttavia, la traslazione è un'operazione *affine*, che richiederebbe un'addizione vettoriale separata ($\mathbf{p}' = R\mathbf{p} + \mathbf{d}$). Per ovviare a questo e manipolare posa (rotazione) e traslazione con una singola e più efficiente operazione moltiplicativa, si proietta lo spazio 3D in uno spazio a 4 dimensioni tramite le **Coordinate Omogenee**.

Si aggiunge una quarta coordinata, detta "peso" o w , ai vettori tridimensionali le quali si dividono in 2 categorie a seconda **del valore della quarta coordinata**:

- **Punti nello spazio:** $\mathbf{p} = [p_x, p_y, p_z, 1]^T$. Il valore "1" indica che il punto possiede una posizione definita e subirà l'effetto di eventuali traslazioni.
- **Direzioni (o vettori puri):** $\mathbf{v} = [v_x, v_y, v_z, 0]^T$. Il valore "0" "spegne" la componente di traslazione della matrice, rendendo il vettore immune agli spostamenti, per esempio il vettore rappresenta una Direzione o un vettore puro (es. una forza, una velocità).

La **Matrice di Trasformazione Omogenea** ${}^A T_B$ di dimensione 4×4 fonde in un'unica entità la matrice di rotazione ${}^A R_B \in SO(3)$ (che descrive l'orientamento del frame B rispetto ad A) e il vettore di traslazione ${}^A O'_B \in \mathbb{R}^3$ (che descrive l'origine del frame B rispetto ad A):

$${}^A T_B = \begin{bmatrix} r_{11} & r_{12} & r_{13} & d_x \\ r_{21} & r_{22} & r_{23} & d_y \\ r_{31} & r_{32} & r_{33} & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} {}^A R_B & {}^A O'_B \\ \mathbf{0}^T & 1 \end{bmatrix}$$

Esempio Pratico e Inversione Rapida:

Immaginiamo di avere uno scanner 3D montato sull'end-effector di un braccio robotico. Il sensore rileva un punto su un pezzo meccanico: $\mathbf{p}_{cam} = [0, 0, 0.5, 1]^T$ (mezzo metro lungo l'asse ottico z). Sappiamo che il frame della telecamera rispetto alla base del robot è traslato di $0.5m$ lungo x e ruotato di 90° ($\pi/2$) attorno all'asse y . La matrice che mappa i punti dalla telecamera alla base sarà:

$${}^{base} T_{cam} = \begin{bmatrix} 0 & 0 & 1 & 0.5 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Moltiplicando ${}^{base} T_{cam} \cdot \mathbf{p}_{cam}$, otteniamo le coordinate del punto lette dal sistema di riferimento della base del robot: $\mathbf{p}_{base} = [1, 0, 0, 1]^T$. Questo approccio matriciale è computazionalmente ideale per trasformare intere *Point Clouds* in un solo colpo moltiplicando la matrice 4×4 per una matrice $4 \times N$ contenente tutti i punti scansionati.

Un vantaggio cruciale delle matrici omogenee è l'efficienza nel calcolo della **matrice inversa** T^{-1} (che mappa da A a B). Invece di eseguire una costosa inversione di Gauss-Jordan su una 4×4 , si sfruttano le proprietà ortogonali della sottomatrice di rotazione ($R^{-1} = R^T$):

$$({}^A T_B)^{-1} = {}^B T_A = \begin{bmatrix} {}^A R_B^T & -{}^A R_B^T \cdot {}^A O'_B \\ \mathbf{0}^T & 1 \end{bmatrix}$$

8 Alberi di Trasformazione (TF Trees)

Un sistema robotico reale non è quasi mai composto da due soli sistemi di riferimento. Un manipolatore su una base mobile equipaggiato con sensori visivi comporta decine di frame (es. `odom`, `base_link`, `link_1` ... `link_6`, `camera_link`, `object_frame`). Mantenere e calcolare manualmente ogni possibile relazione spaziale tra coppie arbitrarie di frame richiede un onere computazionale di $O(N^2)$ e risulta impossibile da gestire dinamicamente.

In architetture moderne come ROS 2 (libreria `tf2`), questo problema viene risolto strutturando i frame in un **TF Tree**, ovvero un Grafo Aciclico Diretto (DAG).

- **Nodi:** Rappresentano i singoli *frame* di coordinate fisiche o logiche del robot.
- **Archi (Edges):** Rappresentano la matrice di trasformazione omogenea ${}^{genitore} T_{figlio}$ che descrive la posa del nodo "figlio" rispetto al nodo "genitore". Un albero valido impone che ogni nodo abbia un solo genitore, garantendo un'unica origine globale (spesso `world` o `map`).

Risoluzione Cinematica su un TF Tree:

Quando un nodo di calcolo (es. MoveIt) necessita di conoscere la trasformazione tra due frame distanti nell'albero, ad esempio da `base_link` (B) a `camera_link` (C), il sistema esegue una ricerca in ampiezza o profondità per trovare l'unico cammino che li collega.

1. **Ricerca del percorso:** Si individua il ramo di connessione, passando per i nodi intermedi (es. i giunti del braccio: $B \rightarrow L_1 \rightarrow L_2 \rightarrow \dots \rightarrow L_6 \rightarrow C$).
2. **Concatenazione (Chain Rule):** Si moltiplicano in sequenza le matrici di trasformazione omogenee lungo il percorso. L'ordine di moltiplicazione (da destra a sinistra per trasformazioni in frame mobili) è fondamentale:

$${}^B T_C = {}^B T_{L_1} \cdot {}^{L_1} T_{L_2} \dots {}^{L_6} T_C$$

3. **Discesa lungo l'albero (Genitore $\rightarrow$ Figlio):** Se il percorso scende lungo un arco (direzione fisiologica del grafo), si utilizza direttamente la matrice salvata in memoria ${}^P T_F$.
4. **Risalita lungo l'albero (Figlio $\rightarrow$ Genitore):** Se il percorso prevede di risalire un ramo prima di scenderne un altro (es. da `camera_link` a `gripper_link` passando per `link_6`), l'algoritmo inverte al volo la matrice dell'arco risalito utilizzando la formula rapida dell'inversa: $({}^P T_F)^{-1} = {}^F T_P$.

Questo approccio disaccoppia la pubblicazione dei dati: i sensori aggiornano solo i rami finali ad alta frequenza (es. `camera` $\rightarrow$ `oggetto`), gli encoder dei motori aggiornano la cinematica interna, e l'algoritmo di navigazione aggiorna la radice, garantendo una fotografia coerente e scalabile di tutto il robot ad ogni istante t .

9 Tabella Riassuntiva delle Rappresentazioni

Metodo	Vantaggi (Pros)	Svantaggi (Cons)	Uso Tipico (ROS)
Matrici di Rot. (3x3)	Niente singolarità, uniche, matematica semplice	Ridondanti (9 var), instabili se modificate	Calcoli interni
Angoli di Eulero	Minimali (3 var), intuitive	Dipendono dall'ordine, singolarità	URDF, interfacce utente
Quaternioni (4D)	Efficienti, niente singolarità, ottimi per SLERP	Matematica complessa, poco intuitivi	ROS Messages
Matrici TF (4x4)	Rappresentazione unificata, computazione rapida	Ridondanti	TF lookups interni